

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

System Name: Purulia Aim Physical Institute (PAPI) Platform	Version: 1.0 (Production Blueprint)
Target Date: 28 September 2026	Status: Approved Architecture Draft
Scope: Public Website + Training Management System + Student Portal + Recruitment Engine + Community	

1. Introduction & Executive Summary

1.1 Purpose: This document specifies the functional, non-functional, security, and architectural requirements for the Purulia Aim Physical Institute (PAPI) platform. It guides implementation across database schemas, APIs, frontend UI, and role-based workflows.

1.2 Background: Purulia Aim Physical Institute trains youth in physical fitness and prepares them for physical entrance examinations for the Indian Army, Navy, Air Force, Police, RPF, CAPF, SSC GD, and related government recruitments. Founded by Anup Kumar Mahato (ex-Army Havaldar), the institute provides free physical training to the youth of Bengal.

1.3 Product Vision: Public Website + Training Management System + Student Portal + Recruitment Information Platform + Community. Communicates **Discipline** (military precision), **Energy** (athletic progress), and **Trust** (transparent data, protected minor records).

1.4 Scope:

- In-Scope (V1):** Public website, admission intake, authentication with RBAC, student telemetry tracking (1600m run, push-ups, long jump, BMI), session-based attendance, recruitment notifications, trainer and admin dashboards, CMS (news, gallery, events, moderated student stories, wall of fame), audit logs.
- Out-of-Scope (V1):** Native mobile applications, real-time WebSocket chat, payment gateway, live GPS runner tracking.

2. Operating Environment & User Classes

Modern mobile and desktop browsers (Android Chrome priority). Responsive down to 360px. Server: Node.js LTS, PostgreSQL 15+, S3/Cloudinary media storage.

Role	Access Level	Core Responsibilities
Visitor	Public Endpoints	Reads recruitment updates, programs, wall of fame; submits applications.
Applicant	No Login	Application submitted; pending admin verification.
Student	Self-Only (Row-Level)	Views personal attendance stats, physical tests, readiness targets; submits stories.
Trainer	Assigned Batches	Records attendance, creates daily sessions, inputs assessment results.
Content Mgr	Content Systems	Moderates student blogs; manages news, events, gallery, and wall of fame.
Admin	Institute Scope	Approves applicants, assigns batches, manages recruitments, reviews reports.
Super Admin	System Wide (2FA)	Configures roles, system settings, views immutable audit logs.

3. Functional Requirements Overview

3.1 Public Portal & Application Intake

- **FR-PUB:** Responsive homepage with video hero, statistics counter, "Free Physical Training for the Youth of Bengal" banner, program eligibility listings (Army, Navy, Police, CAPF, SSC GD), recruitment bulletin, gallery, and contact information.
- **FR-APP:** Multi-step intake form collecting demographic, emergency contact, physical baseline (height, weight, chest), and recruitment goals. Independent consent checkboxes for data storage and publicity; mandatory guardian consent for minors under 18.

3.2 Authentication & Security

- **FR-AUTH:** Stateless JWT with short-lived Access Tokens (15 min) and rotating Refresh Tokens (7 days) stored exclusively in HttpOnly; Secure; SameSite=Strict cookies. Password hashing via Argon2id. Login rate-limiting and account lockouts. Mandatory TOTP 2FA for Super Admin.

3.3 Progress Engine & Attendance

- **FR-PRG:** Stores structured numeric assessments: 1600m run (seconds), push-ups, sit-ups, pull-ups, long jump (cm), high jump (cm), weight (kg), and height (cm). API calculates BMI, personal records, percentage improvements, and gap deltas against official recruitment standards.
- **FR-ATT:** Single-transaction bulk attendance marking (Present, Absent, Late, Leave). Flags students with 3+ consecutive unexcused absences.

4. System Architecture & Tech Stack

- **Style:** Modular Monolith (Clean Architecture).
- **Frontend:** Next.js (App Router), TypeScript, Tailwind CSS, Radix UI (shadcn), Recharts, Framer Motion.
- **Backend:** NestJS (REST API), TypeScript, Prisma ORM, Passport.js.
- **Database & Storage:** PostgreSQL 15+, Cloudinary / S3 for images and documents.

5. Engineering Execution Prompts (Plain-Text Auto-Wrapped)

Copy and run these prompts sequentially with your AI coding tool. Text wrapping is configured for clean print layout.

PROMPT 1: MONOREPO SETUP & WORKSPACE CONFIGURATION

Act as a Principal Software Architect (15+ years experience).
 Project: Purulia Aim Physical Institute (PAPI) Platform.
 Stack: Turborepo, Next.js (App Router), NestJS, PostgreSQL, Prisma, Tailwind CSS, Zod.

TASK:

1. Provide the exact shell commands to scaffold a Turborepo monorepo with:
 - apps/web (Next.js with Tailwind)
 - apps/api (NestJS modular API)
 - packages/validation (Shared Zod schemas)
 - packages/types (Shared domain interfaces)
2. Provide root package.json with workspace configurations.
3. Configure turbo.json with optimized build, test, and lint dependency pipelines.
4. Output the root tsconfig.base.json enforcing strict null checks and module resolution.

RULES:

- Zero boilerplate narrative. Output raw scripts and production configuration code only.
- Strict DRY compliance.

PROMPT 2: RELATIONAL DATABASE SCHEMA & SHARED CONTRACTS

Act as a Database Architect.
 Context: PAPI Platform Postgres Schema via Prisma.

TASK:

1. Write the complete, production-grade schema.prisma.
 - Models: User, Role (Enum), Student, Batch, TrainingSession, Attendance, Assessment, AssessmentResult, TargetStandard, Recruitment, AuditLog.
 - Use UUIDv4 primary keys, explicit foreign key constraints with indexed lookup vectors.
 - Enforce composite unique constraints: @@unique([sessionId, studentId]) on Attendance and @@unique([assessmentId, studentId]) on AssessmentResult.
 - Use smallint and integer data types for metrics (e.g., runTimeSeconds, pushups, longJumpCm) to save storage.
2. In packages/validation, write the reusable Zod validation schemas for:
 - Trainee admission application (multi-step validation including minor guardian consent).
 - Single-transaction bulk assessment recording.
3. Export inferred TypeScript types directly from Zod definitions.

RULES:

- No placeholder comments. Write the full schema.
- Include database-level indexing for high-frequency queries.

PROMPT 3: BACKEND SECURITY, AUTH & RBAC SUBSYSTEM

Act as a Principal Backend Security Engineer.

Context: NestJS API application in apps/api.

TASK:

Write the complete, memory-efficient authentication and authorization subsystem:

1. AuthService:

- Password hashing using argon2id (memoryCost: 65536, timeCost: 3).
- Login issuing a short-lived Access Token (15m) and rotating Refresh Token (7d).
- Write cryptographic tokens directly to secure cookies: HttpOnly; Secure; SameSite=Strict.

2. JwtAuthGuard & RolesGuard:

- Reflective execution reading role requirements via custom @Roles(...) metadata.
- Extraction of JWT from signed cookies (zero Authorization header exposure).

3. AuditInterceptor:

- Interceptor logging mutation events (POST, PATCH, DELETE) to the AuditLog table asynchronously without blocking the client response cycle.

4. Generic NestJS Exception Filter sanitizing raw database errors and returning standardized JSON: { statusCode, error, message, timestamp }.

RULES:

- Pure TypeScript. Avoid unnecessary imports. Keep memory footprint minimal.

PROMPT 4: TELEMETRY CALCULATION ENGINE & BATCH ATTENDANCE

Act as a Senior Backend Systems Developer.

Context: NestJS Telemetry & Attendance Modules.

TASK:

1. Implement AttendanceService:

- Method recordBulkAttendance(sessionId: string, records: { studentId: string, status: AttendanceStatus } []).
- Execute in a single atomic database \$transaction.
- Flag trainees breaching 3 consecutive absences without separate read queries.

2. Implement ProgressEngineService:

- Method calculateStudentTelemetry(studentId: string, targetForce: Force):
- Query last 5 assessments and compute: BMI, % change for 1600m run, push-ups, and pull-ups.
- Fetch benchmark criteria for the selected force from TargetStandard and calculate the readiness gap: { metric, current, target, qualified: boolean, gap: number }.
- Prevent N+1 queries by using Prisma's select projection.

RULES:

- Production-ready code with complete logic. No partial implementations.

PROMPT 5: GENERIC, ACCESSIBLE REUSABLE UI PRIMITIVES

Act as a Senior Frontend UI Architect.

Context: Next.js App Router (apps/web), Tailwind CSS, Radix UI.

TASK:

Build completely generic, fully-typed UI primitives honoring the PAPI brand design tokens:

Primary: Deep Olive (#26351F), Secondary: Army Green (#4B6135), Accent: Saffron (#F59E0B).

1. DataTable<TData, TValue>:

- Headless table wrapper powered by TanStack Table.
- Virtualized rendering for performance with hundreds of student records.
- Dynamic sorting, filtering, and pagination support.

2. FormWrapper:

- Generic form engine integrating react-hook-form and shared Zod schemas.
- Auto-generates field errors and loading states.

3. StatMetricCard:

- Displays performance KPIs: current value, baseline, percentage improvement pill (+/-), and progress bar against benchmark targets.

RULES:

- Zero any types. Provide full TypeScript generics.
- No code duplication across dashboard components.

PROMPT 6: STUDENT PROGRESS VISUALIZER & REPORT EXPORT

Act as a Senior Full-Stack Engineer.

Context: Next.js (apps/web) Trainee Telemetry Portal.

TASK:

1. Build StudentProgressVisualizer:

- Responsive multi-metric chart using Recharts.
- Plot 1600m running times over date sequences with a horizontal reference line for the Army GD passing target.
- Tooltip formatting displaying absolute improvement in seconds and percentage changes.

2. Build ProgressReportPDFView:

- Print-ready and CSS-isolated evaluation sheet component containing student stats, trainer remarks, attendance %, and verification stamp.

3. Optimize page performance to ensure Largest Contentful Paint (LCP) < 2.5s using React Suspense and dynamic imports for heavy charting libraries.

RULES:

- Mobile-first responsiveness (360px viewport upwards).
- Clean code architecture with clear visual separation.